

Scenario 1: a provider onboards a PDF. Nobody has asked anything yet.

```
step 1 ingest the PDF, extract passages ONCE
step 2 in parallel:
    2a passages → Beckn catalogue metadata (small; goes to the network)
    2b passages → embeddings → vector DB (large; stays with the provider)
step 3 /catalog/publish → the network layer
```

The manager document is **STEP2_WALKTHROUGH.md** — a step-by-step runthrough of 2a and 2b with real values from a real run, and the full vector-DB technicality. Read that one. This file is orientation and how to run it.

In plain language first

Onboarding a bulletin is like cataloguing a library. A new bulletin arrives, and you do two unrelated things to it at the same time:

1. You write an **index card** — which crops it covers, which districts, which languages, how often it is refreshed. Small. That card goes out to the network so anyone can find you.
2. You put the **bulletin itself on a shelf** in your own building, arranged so you can find any paragraph inside it later. Nobody outside gets the bulletin.

Nobody has asked you anything yet. You do both because the bulletin was published, and you do it the same way regardless of who eventually asks or what they ask about.

The reason there are two branches at all — rather than one big index — is that the index card **cannot contain the bulletin**. A catalogue is cached by every consumer that fetched it, so advisory text placed inside it goes stale silently and there is no mechanism to recall it. Advice therefore has to be fetched live from the provider, which is exactly what branch 2b exists to serve.

Run it

```
# Python >= 3.10 required. The repo's existing venv already has everything
# except sentence-transformers.
pip install -r requirements.txt

python -m publish_pipeline_beckn_v4.run_scenario1 --all --fresh
pytest publish_pipeline_beckn_v4/tests/test_v4.py -q -m "not semantic" # 29 tests, ~9s
pytest publish_pipeline_beckn_v4/tests/test_v4.py -q -m semantic      # 1 test, needs the model
```

Saved output from a real run is checked in, so the walkthrough can be read without running anything:

- **evidence/SCENARIO1_1_TRANSCRIPT.txt**
- **evidence/publish_payload.json** — the actual `/catalog/publish` body (155 KB compact, 278 KB as pretty-printed here)
- **evidence/resources/** — **one JSON file per bulletin**, each carrying that document's resources with their full `resourceAttributes`. Usually the more useful view: "what did *this* bulletin claim?" without scrolling past two other states.

file	state	resources	size
karnataka.json	IN-KA	61	163 KB
up.json	IN-UP	8	55 KB
rajasthan.json	IN-RJ	4	14 KB

- `evidence/message_update.reference.json` — the target shape, kept alongside so a test can diff against it
- `docs_pdf/` — this file and the walkthrough as PDF

To print a single resource's attributes straight to the terminal (matched on any substring of its id — note the order is `...-<category>-<area>`):

```
python -m publish_pipeline_beckn_v4.run_scenario1 --all --show-resource livestock-in-ka-koppal
```

It runs without the model, and says so

The default is the real `intfloat/multilingual-e5-large` (1024-d, multilingual). `EMBEDDING_BACKEND=lexical` runs with nothing downloaded, but it matches **spellings, not meanings** — no paraphrase, no cross-language — and stamps `semantic=False` on every result it returns. That is deliberate: a plausible-looking similarity score from a bag of character n-grams is the easiest way to mislead a room.

What one real run does

Three real government bulletins, chosen because they differ in ways that show up in the output — `--all --fresh`, on Apple Silicon (`device=mps`):

	Karnataka	Uttar Pradesh	Rajasthan
pages	53	45	30
passages	155	177	25
language mix	en 154 / hi 1	en 62 / hi 115	en 2 / hi 23
subjects resolved	80.6%	45.2%	12.0%
districts resolved	31 / 31	75 / 75	32 / 32
passages placed in a district	97.4%	42.9%	64.0%
2a resources	61	8	4
capability types	4	4	3
2b vectors	155	177	25
max tokens / passage	254	314	305
2a time	~0.01 s	~0.01 s	~0.02 s
2b time	15–186 s	22–154 s	4–49 s

```
step 3  ACCEPTED [ ] 3 catalogues, 73 resources, 154,599-byte payload
        network layer now holds 121,198 bytes of resourceAttributes:
          73 resources [ ] 51 subject URIs [ ] 141 area codes [ ] 13 topics
          [ ] 4 subject categories [ ] 7 weather parameters [ ] 2 languages
        and zero advisory text [ ] checked against real sentences from the source
        round-trip verified: every field held exactly as published

totals  357 passages [ ] 73 resources [ ] 357 vectors [ ] 0 refused
```

Why the three states differ so much is the interesting part, and it is real signal rather than a bug:

- **Karnataka publishes 61 resources from 53 pages** because it is organised as per-district advisory sections (*Agromet Advisory for Koppal district*), so almost every passage lands in a district and becomes part of a district-level capability.
- **UP publishes only 8 resources from more passages (177)** because it is organised by agro-climatic zone, and its tables list 7–8 districts per row. Those passages are honestly statewide, so they group into a handful of state-level resources — while still naming all 75 districts in *coverageAreas*, so a consumer can narrow down.
- **Rajasthan resolves only 12% of passages to a crop** because it genuinely is mostly weather warnings, not crop advisories. The run prints that as a warning rather than letting a thin catalogue look complete.

And the retrieval works

```
query: "pigeon pea flowers dropping, what should I spray"
0.888 imd_karnataka_agromet.pdf p.16 res-...-crop-in-ka-koppal
      "Redgram Flowering Nipping in Pigeon pea at 50 days after sowing...
      Dropping of flower in pigeon pea du..."
```

The word "tur" never appears in that bulletin — it says "Redgram" and "Pigeon pea". A query using "tur" still finds it (there is a test for exactly that), which is what the 1024-d multilingual model is buying. A Hindi query (धान की फसल में सिंचाई) returns Hindi passages from the UP bulletin.

Files

file	what it is	read it for
<code>config.py</code>	every setting, and what each degrades to	"what do I need installed"
<code>taxonomy/vocab.py</code>	crop/district/topic vocabulary + alias resolution	the only thing both branches share
<code>taxonomy/ids.py</code>	resource-id and point-id rules	why a reissue updates instead of duplicating
<code>taxonomy/data/*.json</code>	the vocabulary itself	what we can and cannot recognise
<code>ingest/pdf_text.py</code>	PDF → pages	ingestion, and when it refuses
<code>ingest/language.py</code>	script/language detection + encoding check	the multi-language story
<code>ingest/passages.py</code>	pages → <code>Passage</code> (text and facets in one object)	why 2a and 2b cannot drift
<code>beckn/models.py</code>	pydantic mirrors of <code>beckn.yaml</code>	spec conformance
<code>beckn/resource_attributes.py</code>	facets → <code>resourceAttributes</code>	the heart of 2a
<code>beckn/catalog.py</code>	passages → resources → catalogue	capability typing
<code>beckn/envelope.py</code>	the <code>message_update.json</code> envelope	step 3's payload
<code>beckn/validate.py</code>	spec + coherence + no-prose checks	what we refuse to publish
<code>vectors/embeddings.py</code>	the model, its prefixes, its 512-token limit	the vector DB: model, dims
<code>vectors/store.py</code>	Qdrant collection, payload, filters, ids	the vector DB: schema, filters
<code>network_node.py</code>	stand-in for the network layer	what it holds , and the two-hop shape
<code>publish.py</code>	step 3	validate-then-deliver
<code>scenario1.py</code>	<code>onboard()</code> , <code>publish_all()</code> , branch timings	the orchestration
<code>run_scenario1.py</code>	runnable, narrated	just run it
<code>tests/test_v4.py</code>	30 tests	most claims above, as assertions
<code>tools/md2pdf.py</code>	optional doc → PDF renderer	regenerating <code>docs_pdf/</code>

Nothing here imports from `pipeline.*`, from `pipeline_beckn_v2` or from `publish_pipeline_beckn_v3`. The whole flow reads end to end without following imports into another package.

The design decisions, in one place

- One extraction pass feeds both branches.** A `Passage` carries the text (2b's payload) *and* its resolved facets (2a's raw material), and both branches read the same `Passage.facets()` method. Two extractors would drift, and the failure mode is nasty: discovery routes a question to a resource whose stored passages carry a different filter, so the provider searches inside a resource it just advertised and finds nothing. `test_facet_parity` holds the line.
- The unit of publication is a capability, not a document.** One resource per (*primary coverage area × capability category*) — "crop advisory for Koppal" — not one per PDF and not one per advisory row. Resource ids derive from (*provider, domain, category, area*) and contain nothing about the bulletin issue, so next Friday's reissue **updates** the same resources instead of minting duplicates.

Because the unit is (*area × category*), **@type is a function of the category, not of the document**. One agromet bulletin therefore publishes four different capability types, and each carries only the attributes its type is for: a crop

resource has `agricultureSubjects` and no `weatherParameters`; a forecast resource the reverse. `validate.py` fails a payload where the two disagree.

1. **The catalogue carries metadata only.** No advisory text, not even a snippet. Size is the small reason; staleness is the real one. `validate.py` fails the payload if prose creeps in, and a test greps the real payload for actual sentences from the source bulletins.
2. **`resourceAttributes` is where all the domain work lives — and it is what the network layer holds.** Beckn core leaves it as an open JSON-LD object on purpose (`Attributes`: requires only `@context` and `@type`, `additionalProperties: true`). Everything agriculture-specific goes there: `agricultureSubjects[]` with taxonomy URIs, `coverageAreas[]` as governed codes, `languages[]`, `topics[]`, `weatherParameters[]`, `forecastHorizon`, `updateFrequency`, `geographicGranularity`, plus an `evidence` block.

Stated in the right order: **the network layer holds the full `resourceAttributes`; what it does not hold is document text.**

1. **Areas are governed codes, never invented coordinates.** A polygon nobody surveyed is a polygon we made up, and downstream nothing can tell it from a real boundary. States use real ISO-3166-2 (`IN-KA`). There is a test asserting the payload contains no `coordinates` key at all.
2. **Extraction is rules-first.** The alias table does the work: deterministic, free, auditable, and byte-identical on every re-run — which matters because a churning catalogue republishes for no reason. **No LLM is in the default path, and no code path may mint a subject URI** that the taxonomy did not; `validate.py` checks every `subjectId` against the vocabulary.
3. **Closed-vocabulary fields are validated by membership, not by eyeball.** A topic must *be* a topic from `capabilities.json`. This is strictly stronger than scanning those fields for prose — which matters, because `"Spraying"` is simultaneously a canonical topic name and a word any prose detector would flag.
4. **A document with no usable text layer is refused, not guessed at.** The alternative is a resource claiming coverage it cannot serve.

Known limits — read before demoing

- **Parallelism buys nothing measurable, and the run says so.** The two branches genuinely execute concurrently in a thread pool, but 2a costs ~10 ms against 2b's tens of seconds, so wall clock is simply 2b's time — the measured saving is ~0 (ratio 568x–15,654x across runs). The reason to run them concurrently is architectural (neither branch blocks the other, and either can fail without corrupting the other), **not throughput**. A real speed-up would come from parallelising *within* 2b. `BranchTiming.summary()` prints the ratio rather than claiming a win.
- **2b's timing varies by more than 12x on identical input.** Karnataka's 155 passages measured 14.9 s, 33.5 s, 83.6 s and 186.2 s across four runs on the same laptop — fastest on an idle machine, slowest with the GPU hot and contended straight after a test run. The table above gives ranges for that reason. Quote no single figure; measure on the target hardware, and expect a dedicated GPU to be both faster and far more consistent.
- **`context.domain` is not in Beckn v2.** The spec's `Context` has no `domain` property — v2 replaced it with `schemaContext`. We emit it because `message_update.json` carries it. Context is not `additionalProperties: false`, so it is tolerated, but it is a house extension rather than spec.
- **`CatalogPublishAction` is marked `deprecated: true`** in `core-v2.0.0-lts`, even though `/catalog/publish` remains the documented publish path and the spec offers no replacement.
- **The schema URLs do not resolve.** `https://schemas.openagrinet.global/...` fails DNS today, and `OpenAgriNet/network-specs` contains only a 15-byte README. `@context` is required by the spec, so we emit the intended URL rather than substitute one that happens to resolve.
- **`provider.availableAt` is omitted.** Beckn's `Location` requires a `geo` geometry and we have no surveyed boundary for any state. Coverage is expressed as area codes inside `resourceAttributes` instead. The supplied `message_update.json` has a hand-drawn Karnataka bounding box; we do not reproduce that.

- **District codes are not LGD.** LGD is the correct national scheme and its codes are numeric; we do not have the master loaded, so districts are emitted under `OPENAGRI-DISTRICT` with a readable code we can vouch for. Swapping in LGD is a two-field change in `taxonomy/vocab.py`.
- **Devanagari in these files has a small encoding defect.** The fonts are proper Unicode (Nirmala UI, Mangal), but the producing tool mis-maps a few glyphs: the UP bulletin emits `प्रदि` where `प्रदेश` belongs. Measured rate: **0.7% of Devanagari words in UP, 0.1% in Rajasthan**. Embedding is robust to it; exact-term lookup on an affected word is not. The alias tables absorb the specific corrupted spellings that appear, and `check_devanagari_encoding()` reports density on every run so a genuinely legacy-font file would be caught.
- **Hindi vs Marathi is not distinguished.** Devanagari is shared between them (plus Nepali, Sanskrit, Konkani). We report `hi` and set `ambiguous=True` with the sibling list, rather than shipping a model to guess.
- **Embedded Qdrant ignores payload indexes** and evaluates filters by scanning. Results are correct; latency is not a production figure. `VectorIndex.describe()` says which mode it is in for exactly this reason.
- **No Beckn signatures.** `AuthorizationHeader`, `AckSignatureHeader` and `context.key` are not implemented. Discovery trusts whoever calls it, and scoping a vector search by `resource_ids` **does not authenticate** that the caller was ever given them — resource ids are public. This is the largest gap and whoever builds the production consumer leg must know it.
- **`network_node.py` is a model, not an implementation.** No registry lookup, no signature verification, no multi-provider fan-out; persistence is a dict.
- **Scenario 2 is not built.** The follow-up loop (step 5 of the pipeline diagram) is out of scope here — scenario 1 involves no question. What is proven is that branch 2b's index *can* answer one: `run_scenario1` ends with a retrieval smoke check, and `test_retrieval_can_be_scoped_to_advertised_resources` demonstrates the discovery→filter→answer path.